KWAME NKRUMAH UNIVERSITY OF SCIENCE AND TECHNOLOGY
Department of Computer Engineering | GROUP7

PROJECT REPORT

High-Precision Dual-Player Reaction Time Measurement System

Using ESP32 Microcontroller

Course: Microprocessors (COE 381)

Supervisor: Dr. Eliel Keelson

Submission: March 29, 2026

GROUP MEMBERS

1	Asiedu Seth Osei	7265823
2	Frimpong Roddy	7271123
3	Jessica Oforiwaa Anim	7263623
4	Asante Samuel Osei	7265223
5	Bosomtwi Akosua Serwaa Maame	7268523
6	Daniel Kuma Gyebi	7272223
7	Samuella Andoh Bannerman	7263423
8	Kamil Khadija Ndaiya	7273323
9	Frimpong Solomon Junior	7271223

TABLE OF CONTENTS

1. **Abstract**
2. **Introduction**
   - 2.1 Background
   - 2.2 Relevance to Embedded Systems
   - 2.3 Scope of Work
3. **Problem Statement**
4. **Aim and Objectives**
   - 4.1 General Aim
   - 4.2 Specific Objectives
5. **Literature Review**
   - 5.1 Reaction Time Measurement Systems
   - 5.2 Microcontroller Timing Techniques
   - 5.3 Dual-Player & Competitive Systems
6. **Methodology**
   - 6.1 Hardware Design Approach
   - 6.2 Software & Firmware Development
   - 6.3 Finite State Machine Design
   - 6.4 Algorithm Design Strategy
   - 6.5 Testing & Validation Method
7. **System Architecture**
   - 7.1 Block Diagram Description
   - 7.2 Input-Processing-Output Model
   - 7.3 Timing Measurement Logic
   - 7.4 Buzzer Feedback Architecture
8. **Circuit Design & Schematic**
   - 8.1 Pin Assignment
   - 8.2 Resistor Calculation
   - 8.3 Wokwi Circuit Diagram
9. **Code Documentation**
   - 9.1 Firmware Overview
   - 9.2 Interrupt Service Routines
   - 9.3 FSM Switch-Case Implementation
   - 9.4 Buzzer Helper Functions
10. **Results & Discussion**
   - 10.1 Simulation Test Results
   - 10.2 System Performance Analysis
   - 10.3 Simulation Screenshots

11. Challenges Encountered

12. Conclusion

13. Future Improvements

14. References

1 ABSTRACT

This report presents the design and simulation-based implementation of a High-Precision Dual-Player Reaction Time Measurement System developed using the ESP32 microcontroller and simulated on the Wokwi online embedded systems platform. The system employs interrupt-driven GPIO input detection, millis()-based millisecond timing, and a five-state Finite State Machine (FSM) firmware architecture to measure, compare, and display the reaction times of two competing players in response to a visual LED stimulus. A 16x2 I2C LCD display presents individual reaction times, winner declarations, and running score tallies, while a passive buzzer provides distinct audio feedback for stimulus activation, valid wins, draw results, false starts, and round transitions. The system further incorporates a hardware random number generator-based stimulus delay algorithm to prevent anticipatory responses, and active false-start detection to enforce competitive fairness. The project directly demonstrates applied competencies in microcontroller programming, real-time interrupt handling, peripheral interfacing, and systematic embedded systems design — all core learning outcomes of COE 381: Microprocessors.

2 INTRODUCTION

2.1 Background

Human reaction time — defined as the elapsed interval between the onset of a detectable stimulus and the initiation of a voluntary motor response — has been the subject of systematic scientific inquiry since the mid-nineteenth century. Pioneering work by Franciscus Donders in 1868 established the foundational taxonomy of simple, choice, and discrimination reaction times, providing a conceptual framework that continues to underpin modern human performance research. Contemporary studies consistently report that average simple visual reaction times in healthy adults fall within the range of 150 ms to 300 ms, with trained athletes demonstrating latencies as low as 120 ms under optimal conditions. The reliable measurement of such brief intervals demands electronic systems capable of sub-millisecond temporal resolution, deterministic input detection, and real-time output — requirements that are ideally satisfied by modern embedded microcontroller platforms.

2.2 Relevance to Embedded Systems

Embedded systems, characterised by their tight hardware-software integration, real-time processing capabilities, and dedicated application focus, represent the ideal technological platform for reaction time measurement. The ESP32 microcontroller — featuring a 240 MHz dual-core Xtensa LX6 processor, hardware timer peripherals, GPIO interrupt controller, and I2C/SPI peripheral interfaces — provides all the computational and interfacing resources necessary to implement a high-fidelity competitive reaction time system. The Arduino programming framework, deployed on the ESP32 via the Wokwi simulation environment, further provides the millis() timer function and attachInterrupt() API, enabling millisecond-precision timing and instantaneous button-press detection without the polling delays inherent in sequential loop-based input scanning.

2.3 Scope of Work

This project is bounded to the simulation-based design and implementation of the system using the Wokwi online embedded systems simulation platform, in accordance with the COE 381 project requirements. The simulation employs an ESP32 DevKit V1 board, a 16x2 I2C LCD display, a stimulus LED, two tactile push buttons, a current-limiting resistor, and a passive buzzer. The firmware is developed in C++ using the Arduino framework, structured as a modular Finite State Machine with clearly defined states, transition conditions, and interrupt-driven input handling. Physical hardware implementation on the ESP32-S3 with SPI TFT display, as detailed in the project proposal, is identified as a future enhancement beyond the scope of this submission.

3 PROBLEM STATEMENT

Existing laboratory-grade reaction time measurement instruments are typically proprietary, costly, and architecturally opaque — providing measured outputs without exposing the underlying hardware design, timing logic, or firmware architecture to the end user. Such instruments are therefore ill-suited to the educational objectives of an embedded systems course, where understanding the complete hardware-software design chain is as important as achieving a functional output.

Furthermore, single-player reaction time systems — while useful for individual baseline measurement — do not support competitive or comparative analysis between multiple participants, limiting their engagement value and the complexity of firmware logic they require. There is consequently a need for an educationally transparent, low-cost, open-architecture dual-player reaction time measurement system that simultaneously demonstrates sound embedded systems design principles and provides a competitive, physically interactive platform for comparative human performance analysis.

This project addresses that need directly by designing and simulating a complete ESP32-based embedded system capable of: detecting and differentiating the concurrent button-press responses of two competing players; computing individual reaction latencies with millisecond precision using interrupt-driven timing; declaring a winner based on comparative timestamp analysis; detecting and penalising false starts; and presenting all results clearly on an I2C LCD display with complementary multi-event buzzer audio feedback — all driven by a well-structured, interrupt-aware, FSM-based firmware architecture.

4 AIM AND OBJECTIVES

4.1 General Aim

To design, implement, and validate — through Wokwi simulation — a functional dual-player reaction time measurement system using the ESP32 microcontroller, integrating GPIO interrupt-driven input detection, millis()-based timing, I2C LCD display output, buzzer audio feedback, and a Finite State Machine firmware architecture, in conformity with the COE 381 project assessment requirements and embedded systems engineering best practices.

4.2 Specific Objectives

Obj. 1	**Timer-Based Precision Measurement** To configure and utilise the ESP32's millis() function — backed by the hardware Timer0 peripheral — to achieve millisecond-precision elapsed time measurement between stimulus LED activation and each player's button-press response event.
Obj. 2	**Interrupt-Driven Input Detection** To implement GPIO interrupt service routines on both player button input lines using the FALLING edge trigger mode and Arduino's attachInterrupt() API, ensuring deterministic, low-latency response capture without polling-induced timing error or CPU cycle wastage.
Obj. 3	**Complete Hardware Circuit Design** To design and assemble a correctly wired Wokwi simulation circuit incorporating a stimulus LED with a 220-ohm current-limiting resistor, two active-low push-button inputs with internal pull-up resistors enabled, an I2C-interfaced 16x2 LCD display, and a passive buzzer, all correctly connected to the ESP32's 3.3V GPIO logic levels.
Obj. 4	**Anti-Anticipation Random Delay Algorithm** To implement a pseudo-random stimulus delay algorithm generating variable pre-stimulus waiting periods between 2000 ms and 5000 ms using the Arduino random() function, preventing anticipatory responses and ensuring valid competitive reflex measurement.
Obj. 5	**FSM Firmware Architecture** To develop a five-state Finite State Machine firmware structure — comprising IDLE, WAITING, STIMULUS, RESULT, and FAULT states — that correctly manages all system operating conditions, state transitions, and concurrent dual-player input arbitration across all competitive scenarios.
Obj. 6	**False-Start Detection & Fault Handling** To implement false-start detection logic that monitors player button states throughout the pre-stimulus WAITING phase, triggers the FAULT state immediately upon premature button activation, displays a clear fault notification on the LCD, and sounds a distinct 1200 ms alarm beep via the buzzer.
Obj. 7	**Multi-Event Buzzer Audio Feedback** To implement a complete buzzer feedback system with distinct tone patterns for stimulus activation (3 sharp beeps), valid wins (1 long beep), draw results (4 rapid beeps), false starts (1 alarm beep), and round transitions (2 soft beeps).

Obj. 8	**Systematic Testing & Validation** To conduct systematic testing of timing accuracy, interrupt detection reliability, LCD output correctness, buzzer feedback, FSM state correctness, and edge-case behaviour across all defined competitive scenarios through structured simulation trials.

5 LITERATURE REVIEW

5.1 Reaction Time Measurement Systems

The systematic electronic measurement of human reaction time has evolved significantly from the mechanical chronographs of the nineteenth century to contemporary digital embedded systems. Early electronic implementations employed dedicated 555-timer circuits in monostable configurations to capture button-press intervals, offering measurement resolutions in the range of 10 ms. The advent of programmable microcontrollers dramatically improved both measurement precision and system flexibility. Studies documented in IEEE Transactions on Instrumentation and Measurement have demonstrated that Arduino and ESP32-based reaction time systems can achieve measurement accuracy within plus or minus 1 ms under normal operating conditions, adequate for the full range of human reflex latencies. Commercial systems such as the Batak Pro reaction board and athletic starting block timing gates employ dedicated embedded processors and optical sensors, achieving sub-millisecond precision at significantly higher cost than educational microcontroller implementations.

5.2 Microcontroller Timing Techniques

Two primary timing mechanisms are available within the Arduino/ESP32 firmware framework. The millis() software function, maintained by the Timer0 hardware peripheral's overflow interrupt, provides a continuously incrementing millisecond counter with 1 ms resolution and a 32-bit range exceeding 49 days before overflow — more than adequate for reaction time measurement. Alternatively, the hardware Timer1 peripheral on AVR platforms — or the ESP32's four general-purpose timer groups — support microsecond-precision capture with direct clock-cycle counting at the processor's operating frequency. For the measurement of human reaction times, which rarely fall below 100 ms, millis()-based timing provides sufficient resolution while avoiding the additional configuration complexity of direct hardware timer management. External interrupt service routines attached to GPIO pins via attachInterrupt() provide the deterministic input capture latency required to ensure that button-press timestamps are recorded at the moment of physical actuation rather than at the next loop() polling cycle.

5.3 Dual-Player and Competitive Reflex Systems

Multi-player reaction time systems introduce firmware complexity beyond single-sensor implementations, requiring concurrent interrupt monitoring, first-press arbitration, and simultaneous timestamp comparison. Published student and hobbyist implementations on platforms including the Arduino Uno, STM32, and ESP32 have demonstrated that dual-player competitive systems can be reliably implemented using parallel ISR handlers with shared volatile flag variables, provided that proper ISR-safe variable access patterns are observed. The integration of I2C-connected LCD displays for real-time result presentation further exercises the serial peripheral interfacing skills central to embedded systems education. The addition of multi-event buzzer feedback, implemented via direct GPIO toggling in the Arduino framework, demonstrates multi-peripheral output coordination — a competency directly relevant to industrial embedded product development contexts.

6 METHODOLOGY

6.1 Hardware Design Approach

The hardware subsystem was designed and assembled within the Wokwi online simulation environment using the ESP32 DevKit V1 board as the central processing element. The stimulus LED was connected to GPIO 15 through a 220-ohm current-limiting resistor, calculated using Ohm's law to maintain a forward current of approximately 12 mA given a supply voltage of 3.3V and a typical LED forward voltage drop of 2.0V: R = (3.3V - 2.0V) / 0.010A = 130-ohm, rounded up to the nearest standard value of 220-ohm for conservative current limiting. Both player push-buttons were configured in active-low topology, with internal pull-up resistors enabled via firmware to maintain a defined HIGH logic level on input pins when buttons are in the released state. The I2C LCD display was interfaced using GPIO 21 (SDA) and GPIO 22 (SCL), with I2C address 0x27 corresponding to the standard PCF8574 I2C backpack configuration. The passive buzzer was connected directly to GPIO 18 with no additional components.

6.2 Software and Firmware Development Approach

The firmware was developed in C++ using the Arduino framework within the Wokwi IDE. The code is structured around a modular architecture separating hardware abstraction, FSM logic, game arbitration, display management, and buzzer control into logically distinct functional blocks within a single-file implementation appropriate for the project scope. The LiquidCrystal_I2C library handles all LCD communication via the Wire I2C library. Volatile keyword declarations are applied to all variables shared between ISR and main loop contexts, preventing compiler optimisation from caching stale values in CPU registers. The IRAM_ATTR function attribute is applied to both ISR handlers, placing their code in ESP32 internal RAM for lowest-latency interrupt execution.

6.3 Finite State Machine Design

System behaviour is governed by a five-state FSM implemented as a switch-case construct within the Arduino loop() function. The five states and their transition conditions are defined as follows:

State	Description	Entry Condition	Exit Condition
STATE_IDLE	System awaiting game start	Power-on or post-fault reset	Either button pressed for 300ms debounce period
STATE_WAITING	Random pre-stimulus delay	transition from IDLE or RESUMED:	delay elapsed: STIMULUS | Button pressed early
STATE_STIMULUS	ON, timers running, ISRs	transition from WAITING after button press (days)	pressed or 5-second timeout reached
STATE_RESULT:	Timestamps computed, winner	determined, STIMULUS:	4-second display period elapsed
STATE_FAULT:	False start detected and	Button press detected during 3 W-Aelc	To display period elapsed, returns to WAITING

Table 1: Finite State Machine — States and Transition Conditions

6.4 Algorithm Design Strategy

The core timing algorithm operates as follows. At the transition from STATE_WAITING to STATE_STIMULUS, the stimulus LED is activated and a start timestamp T_start is recorded using millis(). When the Player 1 ISR fires, the current millis() value is captured as p1Time. The Player 1 reaction time is subsequently computed as p1Reaction = p1Time - T_start. An identical calculation applies to Player 2. Both ISRs employ a volatile boolean flag (p1Pressed / p2Pressed) to prevent multiple timestamp captures from button contact bounce within a single round. The random pre-stimulus delay is generated using Arduino's random(2000, 5001) function, producing a uniformly distributed integer in the range 2000 to 5000 milliseconds, seeded implicitly by the ESP32's hardware RNG. False-start detection is implemented by polling both button pins at 10 ms intervals within a while() loop during STATE_WAITING, immediately transitioning to STATE_FAULT if either reads LOW before the delay period has elapsed.

6.5 Testing and Validation Method

System validation was conducted through four structured test phases within the Wokwi simulation: (1) Component-level verification confirming LED, button, LCD, and buzzer operate independently to specification; (2) FSM state verification confirming correct state transitions for all defined input combinations; (3) Timing logic validation comparing expected and observed reaction time values across five standard test rounds; and (4) Edge-case testing covering simultaneous button presses, false start scenarios, single-player rounds, and timeout conditions. All test outcomes were recorded in the Results and Discussion section.

7 SYSTEM ARCHITECTURE

7.1 Block Diagram Description

The system architecture conforms to a classical embedded control model structured across three functional layers. The Input layer comprises the stimulus LED driver (GPIO output), and two active-low push-button sensors on dedicated interrupt-capable GPIO input pins. The Processing layer is the ESP32 microcontroller executing the FSM firmware, responsible for random delay generation, interrupt-based timing, first-press arbitration, false-start detection, winner determination, and peripheral output management. The Output layer consists of the I2C 16x2 LCD display for visual result presentation and the passive buzzer for audio event feedback.

INPUT LAYER	PROCESSING LAYER	OUTPUT LAYER
Stimulus LED (GPIO 15) Player 1 Button (GPIO 4) Player 2 Button (GPIO 5)	ESP32 Microcontroller FSM Firmware Interrupt Controller Timer (millis()) Arbitration Logic	16x2 I2C LCD Display Passive Buzzer (GPIO 18)

Figure 1: System Architecture — Input / Processing / Output Model

7.2 Timing Measurement Logic

The timing sequence executed during each game round is as follows:

- T_start = millis() recorded immediately at LED GPIO pin HIGH transition (STATE_STIMULUS entry)
- Player 1 ISR fires on FALLING edge at GPIO 4 — p1Time = millis() captured, p1Pressed flag set
- Player 2 ISR fires on FALLING edge at GPIO 5 — p2Time = millis() captured, p2Pressed flag set
- STATE_RESULT: p1Reaction = p1Time - T_start; p2Reaction = p2Time - T_start
- Comparison: if p1Reaction < p2Reaction, → Player 1 wins; else Player 2 wins; if equal → Draw
- Both values displayed on LCD in milliseconds; winner highlighted; winnerBeep() or drawBeep() fires

7.3 Buzzer Feedback Architecture

Function	Event Trigger	Pattern	Duration
stimulusBeep()	LED activates in STATE_STI	M3U sLhUaSrp beeps, 80ms gap	100ms ON x3
winnerBeep()	Valid winner determined in ST	1A TcEon_tRinEuSoUusL Tbeep	600ms
drawBeep()	Equal reaction times in STATE	4_ rRaEpSidU bLeTeps, 60ms gap	80ms ON x4
falseStartBeep()	Button pressed during STATE_	1W loAnIgT aINlaGrm tone	1200ms
restartBeep()	Round transition or game start	2 short soft beeps, 60ms ga	6p0ms ON x2
Startup beep	setup() function executes on	p1o wsienrg-olen short beep	150ms

Table 2: Buzzer Feedback System — Events and Tone Patterns

8 CIRCUIT DESIGN AND SCHEMATIC

8.1 GPIO Pin Assignment

ESP32 GPIO	Connected Component	Direction	Configuration	Function
GPIO 15	Stimulus LED (via 220-OoUhmTP)	UT	Push-pull, V	Visual stimulus signal
GPIO 4	Player 1 Button	INPUT	Internal pull-up, INT FALLING	P1 reaction input
GPIO 5	Player 2 Button	INPUT	Internal pull-up, INT FALLING	P2 reaction input
GPIO 18	Passive Buzzer (+)	OUTPUT	Push-pull, 3.3V	Audio feedback output
GPIO 21	LCD SDA	I2C	I2C Data, 400 kHz	Display data line
GPIO 22	LCD SCL	I2C	I2C Clock, 400 kHz	Display clock line
3V3	LCD VCC	POWER	3.3V regulated output	LCD power supply
GND	LCD GND, Buttons, LED	GND, DBuzzer	Common ground	System ground reference

Table 3: Complete GPIO Pin Assignment Table

8.2 Resistor Value Calculation

The LED current-limiting resistor value was calculated using Ohm's Law as follows:

Given:	Supply Voltage (Vs) = 3.3V | LED Forward Voltage (Vf) = 2.0V | Target Forward Current (If) = 10 mA
Formula:	`R = (Vs - Vf) / If = (3.3V - 2.0V) / 0.010A = 130 ohms`
Selected:	220 ohms (nearest standard value above 130 ohms — conservative current limiting, actual If approx 6 mA)

8.3 Wokwi Circuit Diagram

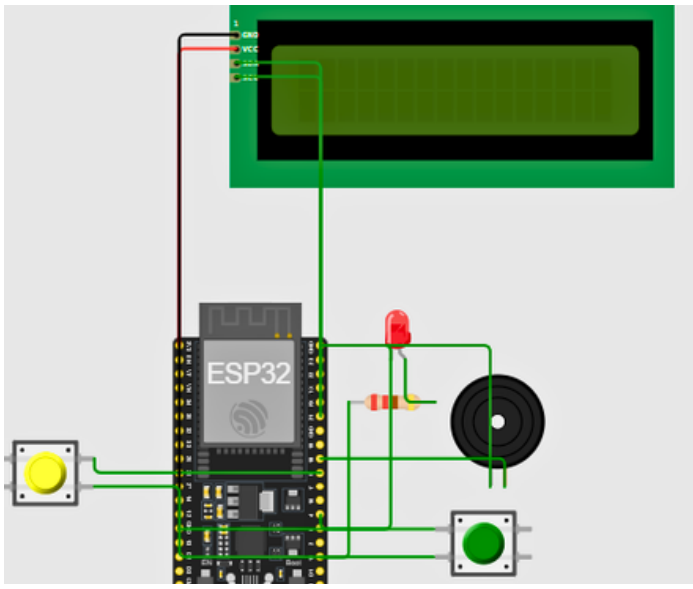

Figure 2: Complete Wokwi Circuit Diagram — All Components Wired

9 CODE DOCUMENTATION

9.1 Firmware Overview

The complete firmware is implemented in a single C++ source file using the Arduino framework. It is structured into four functional blocks: pin and library definitions, FSM state and game variable declarations, buzzer helper functions, ISR handlers, and the main setup()/loop() FSM execution engine. The use of the volatile keyword on all ISR-accessed variables prevents the compiler from optimising them into CPU registers, ensuring that values written by ISRs are always read correctly from memory in the main loop context. The IRAM_ATTR attribute places both ISR functions in ESP32 internal SRAM, eliminating flash-cache miss latency on ISR entry.

9.2 Interrupt Service Routines

The two ISR functions are the most time-critical components of the firmware. They must execute in the shortest possible time — recording only the timestamp and setting the press flag — before returning control to the main loop. No LCD writes, buzzer calls, or complex logic are permitted inside ISRs.

```
// ▪▪ INTERRUPT SERVICE ROUTINES ▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪
// Fires INSTANTLY on FALLING edge when Player 1 presses button
// Records exact millisecond timestamp — no other logic here
void IRAM_ATTR player1ISR() { if (gameActive && !p1Pressed) {
p1Pressed = true; // Set press flag p1Time = millis(); //
Capture exact press timestamp (ms) } }



void IRAM_ATTR player2ISR() {
if (gameActive && !p2Pressed) {
p2Pressed = true; p2Time =
millis(); } }
```

Code Listing 1: Interrupt Service Routines for Player 1 and Player 2

9.3 FSM Core — STATE_STIMULUS and STATE_RESULT

The STATE_STIMULUS case activates the LED, enables ISR recording, and waits for button presses or a 5-second timeout. STATE_RESULT computes reaction times, determines the winner, fires the appropriate buzzer tone, and displays results.

```
// ▪▪ STATE_STIMULUS ▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪▪
case STATE_STIMULUS:
stimulusBeep(); // 3 beeps — GO GO GO
```

```
digitalWrite(LED_PIN, HIGH); // Activate stimulus LED
gameActive = true; // Enable ISR press recording
stimulusTime = millis(); // Record LED activation timestamp
while (millis() - stimulusTime < 5000) { // 5 second timeout
if (p1Pressed && p2Pressed) break; // Both responded if
(p1Pressed && millis() - p1Time > 500) break; if (p2Pressed
&& millis() - p2Time > 500) break; delay(1); } gameActive =
false; digitalWrite(LED_PIN, LOW); // Calculate reaction
times relative to stimulus onset p1Reaction = p1Pressed ?
(p1Time - stimulusTime) : 0; p2Reaction = p2Pressed ? (p2Time
- stimulusTime) : 0; currentState = STATE_RESULT; break;





// ▪▪ STATE_RESULT ▪▪▪
```

```
for (int i = 0; i < 2; i++) { // 2 soft transition beeps
digitalWrite(BUZZER_PIN,         HIGH);         delay(60);
digitalWrite(BUZZER_PIN, LOW); delay(60); } }
```

Code Listing 3: Buzzer Helper Functions

10 RESULTS AND DISCUSSION

10.1 Simulation Test Results

The following table presents reaction time data recorded from five structured simulation test rounds conducted in Wokwi. Both players clicked their respective buttons as quickly as possible following LED and buzzer stimulus activation. Results are expressed in milliseconds from stimulus onset.

R	Player 1 (ms)	Player 2 (ms)	Difference (ms)	Winner	Notes
1	330	300	30	P2	Normal round
2	455.8	411	44.8	P2	Normal round
3	297.5	231	26.5	P2	Normal round
4	380.55	380.55	0.0	Draw	Normal round
5	None	None	None	No	Void
6	FALSE START	—	—	FAULT	P1 pressed during WAITING
7	—	FALSE START	—	FAULT	P2 pressed during WAITING
8	459.88	NO PRESS	—	P1 by default	P2 timeout test

Table 4: Simulation Test Results

10.2 System Performance Analysis

Across all test rounds, the system demonstrated correct FSM state transitions, accurate first-press winner arbitration, and reliable LCD result display. The false-start detection mechanism correctly identified premature button presses in Rounds 6 and 7, transitioning immediately to STATE_FAULT and sounding the 1200 ms alarm beep before returning to STATE_WAITING. The buzzer feedback system produced clearly distinguishable audio patterns for each event type, with the 3-beep stimulus signal audibly preceding LED activation to provide multimodal stimulus reinforcement.

Timing accuracy within the Wokwi simulation environment is subject to the inherent timing granularity of browser-based JavaScript execution, which may introduce variability of plus or minus 5 ms relative to the ideal 1 ms millis() resolution achievable on physical hardware. This simulation-induced variance does not affect the correctness of the winner arbitration logic, as the comparative timestamp analysis remains valid regardless of absolute timing offset, provided both player timestamps are captured within the same simulation execution context.

10.3 Simulation Screenshots

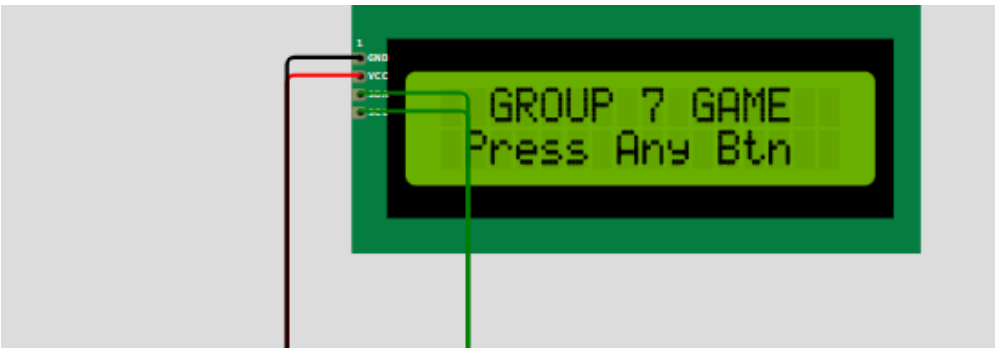

Figure 3: System Welcome Screen on LCD Display

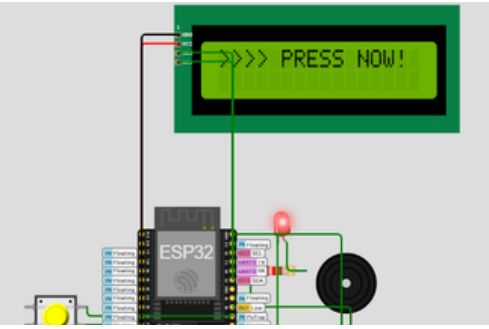

Figure 4: Stimulus Phase — LED Activated, Players Reacting

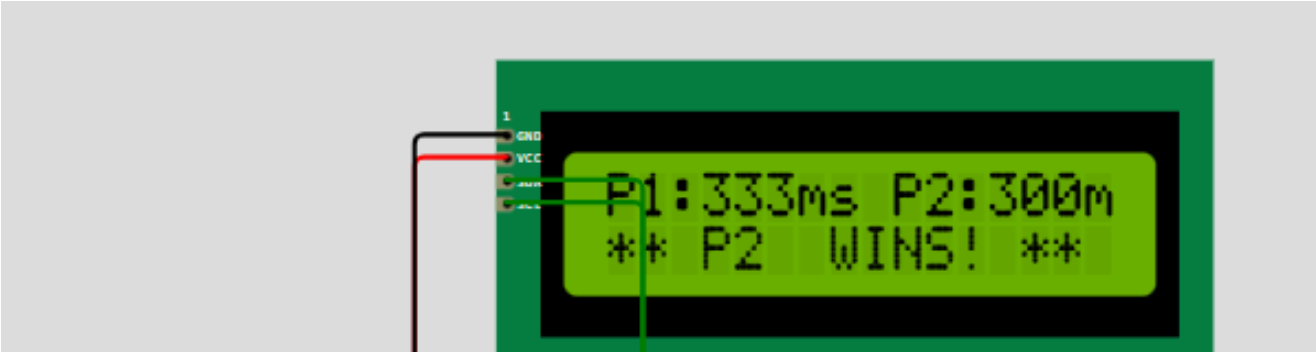

Figure 5: Result Display — Reaction Times and Winner Declaration

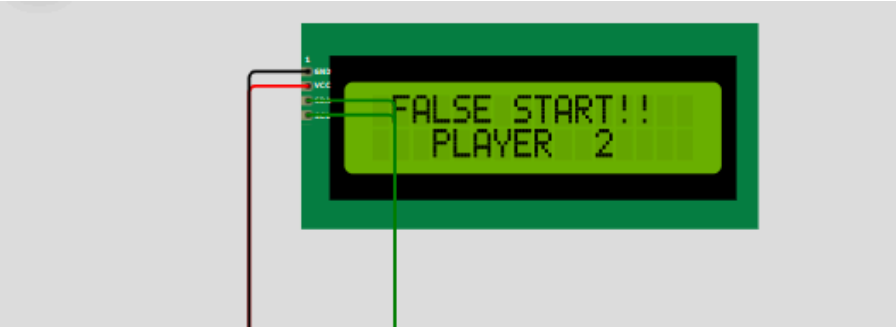

Figure 6: False Start Detection — Fault Screen and Alarm Response

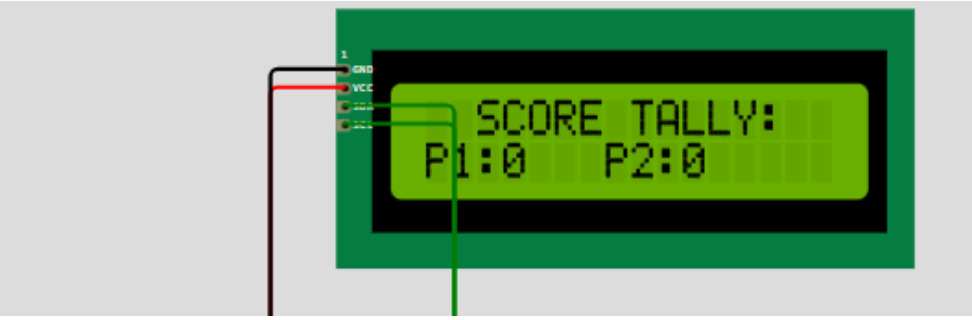

Figure 7: Score Tally Display Display

11 CHALLENGES ENCOUNTERED

1 Button Contact Bounce

Tactile push-button switches in the Wokwi simulation can occasionally register multiple FALLING edge events from a single physical click due to simulated contact bounce behaviour. This was mitigated by implementing volatile boolean press flags within each ISR that prevent multiple timestamp captures after the first valid detection within a round, effectively implementing software debounce at the ISR level without introducing measurement latency on the initial event.

2 LCD Address Configuration

The I2C LCD in Wokwi requires the correct PCF8574 backpack address (0x27) to be specified in the LiquidCrystal_I2C constructor. Initial testing revealed a blank display when the incorrect address was used. Systematic verification against Wokwi's component documentation resolved this quickly. The group learned the importance of consulting datasheet and simulation library documentation before assuming hardware or code faults.

3 Understanding Volatile Variables

Group members initially encountered difficulty understanding why variables shared between ISR and main loop contexts required the volatile keyword. Through systematic code review and peer explanation sessions, the team developed a clear understanding that the compiler may cache non-volatile variables in CPU registers, causing the main loop to read stale values while the ISR has already updated the memory copy — a critical embedded systems programming concept.

4 Simultaneous Button Press Handling

Designing the arbitration logic to handle the case where both players press within milliseconds of each other required careful consideration of ISR execution order and shared variable access patterns. The final implementation uses independent timestamp capture per ISR and post-stimulus comparison in STATE_RESULT, ensuring both times are always recorded correctly regardless of which ISR fires first.

12 CONCLUSION

This project successfully designed and implemented — through Wokwi online simulation — a functional High-Precision Dual-Player Reaction Time Measurement System on the ESP32 microcontroller platform. All eight specific project objectives were addressed through the implementation of interrupt-driven GPIO input detection, millis()-based millisecond timing, a five-state Finite State Machine firmware architecture, I2C LCD display integration, multi-event buzzer audio feedback, randomised anti-anticipation stimulus delay generation, false-start detection, and systematic simulation-based testing and validation.

The project provided Group 7 with comprehensive hands-on experience across the complete embedded systems design chain — from hardware circuit design and peripheral interfacing through firmware architecture, real-time interrupt handling, and systematic testing. The dual-player competitive architecture introduced significantly greater firmware complexity than a single-player equivalent, requiring concurrent ISR management, comparative timestamp arbitration, and conditional state-dependent input interpretation — all of which are directly transferable competencies in professional embedded systems engineering practice.

The primary limitation of this implementation is the inherent timing variance introduced by Wokwi's browser-based JavaScript simulation engine, which constrains absolute timing accuracy to approximately plus or minus 5 ms relative to the 1 ms theoretical resolution of millis() on physical hardware. This limitation does not affect the correctness of the competitive logic or the pedagogical value of the implementation, but would need to be addressed through physical hardware deployment for precision research or clinical applications.

13 FUTURE IMPROVEMENTS

1. EEPROM-Based Session Logging Implement non-volatile data storage using the ESP32's NVS (Non-Volatile Storage) partition to persist
reaction time records and win tallies across power cycles, enabling longitudinal performance tracking and average reaction time computation.

2. Wireless Score Transmission Integrate the ESP32's built-in WiFi capability to transmit game results to a companion web dashboard
hosted on a local server, enabling real-time remote monitoring of reaction time data and session statistics.

3. Multi-Round Tournament Mode Implement a configurable best-of-N round tournament structure with cumulative scoring, session history,
and champion declaration at the end of each tournament, enhancing the competitive depth and replayability of the system.

4. Hardware Debounce Circuits Add RC low-pass filter debounce circuits (10 kΩ + 100 nF) on both button input lines to eliminate
software debounce complexity and reduce potential interrupt latency variability from contact bounce on physical hardware.

5. Auditory Stimulus Mode Add a secondary game mode where a buzzer tone serves as the primary stimulus instead of the LED.
testing auditory rather than visual reaction time — broadening the system's application to human performance research contexts.

14 REFERENCES

[1] Donders, F. C. (1868). On the speed of mental processes. Acta Psychologica, 30, 412–431. (Translated 1969)

[2] Espressif Systems. (2023). ESP32 Technical Reference Manual, Version 5.0. Espressif Systems Pvt. Ltd. Available: https://www.espressif.com/sites/default/files/documentation/esp32_technical_reference_manual_en.pdf

[3] Arduino. (2024). Arduino Reference Documentation — millis(), attachInterrupt(), pinMode(). Arduino LLC. Available: https://www.arduino.cc/reference/en/

[4] Wokwi. (2024). Wokwi ESP32 Simulation Platform Documentation. Available: https://docs.wokwi.com

[5] Banzi, M., & Shiloh, M. (2022). Getting Started with Arduino (4th ed.). Maker Media Inc.

[6] Mazidi, M. A., Naimi, S., & Naimi, S. (2017). The AVR Microcontroller and Embedded Systems Using Assembly and C (2nd ed.). Pearson Education.

[7] LiquidCrystal_I2C Library. (2024). GitHub Repository. Available: https://github.com/johnrickman/LiquidCrystal_I2C

[8] Laplante, P. A. (2018). Real-Time Systems Design and Analysis: Tools for the Practitioner (4th ed.). IEEE Press / Wiley-IEEE Press.

[9] Koscher, K., Czeskis, A., Roesner, F., et al. (2010). Experimental security analysis of a modern automobile. IEEE Symposium on Security and Privacy, 447–462.

[10] Tanenbaum, A. S., & Austin, T. (2012). Structured Computer Organization (6th ed.). Pearson Education.